



National Taiwan University

TokidokiBosottoSegukideNaguru TonarinoSamusan

FHVirus, nathanlee726, Ji_Kuai

2024 ICPC Taichung

Nov 17, 2024

```
1 #define debug(...) ((void)0)
  #define print(...) ((void)0)
  #endif
3 troubleshoot.txt
52 lines

Pre-submit:
4 Write a few simple test cases if sample is not enough.
  Are time limits close? If so, generate max cases.
  Is the memory usage fine?
8 Could anything overflow?
  Make sure to submit the right file.

10 Wrong answer:
  Print your solution! Print debug output, as well.
14 Are you clearing all data structures between test cases?
  Can your algorithm handle the whole range of input?
  Read the full problem statement again.
16 Do you handle all corner cases correctly?
  Have you understood the problem correctly?
  Any uninitialized variables?
21 Any overflows?
  Confusing N and M, i and j, etc.?
  Are you sure your algorithm works?
23 What special cases have you not thought of?
```

Are you sure the big functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.

```

ines
\
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

ines
on
Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0, for example)

```

```
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

```
lines
cmp.sh
6 lines
#!/usr/bin/env bash
set -e
while :; do
    python3 gen.py > input
    diff -y <(.bru < input) <(.sol < input)
done
```

2.1 Equations

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

where A'_i is A with the i 'th column replaced by b .

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \dots, d_k$ s.t.

Non-distinct roots r become polynomial factors, e.g.
 $a_n = (d_1 n + d_2) r^n$.

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\sin v + \sin w = 2 \sin \frac{v+w}{2} \cos \frac{v-w}{2}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}$, $\phi = \text{atan2}(b, a)$.

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

$$\text{Area: } A = \sqrt{p(p-a)(p-b)(p-c)}$$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):

$$m_a = \frac{1}{2} \sqrt{2b^2 + 2c^2 - a^2}$$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b+c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan \frac{\alpha+\beta}{2}}{\tan \frac{\alpha-\beta}{2}}$

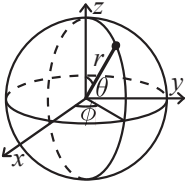
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = \sqrt{4e^2f^2 - F^2} = F|\tan \theta| \ (\theta \neq \frac{\pi}{2})$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Spherical coordinates



$$x = r \sin \theta \cos \phi$$
$$y = r \sin \theta \sin \phi$$
$$z = r \cos \theta$$

$$r = \sqrt{x^2 + y^2 + z^2}$$
$$\theta = \operatorname{acos}(z/\sqrt{x^2 + y^2 + z^2})$$
$$\phi = \operatorname{atan2}(y, x)$$

2.5 Derivatives/Integrals

$$\frac{d}{dx} \arcsin x = \frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx} \tan x = 1 + \tan^2 x$$
$$\int \tan ax = -\frac{\ln|\cos ax|}{a}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x)$$

$$\frac{d}{dx} \arccos x = -\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx} \arctan x = \frac{1}{1+x^2}$$
$$\int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2}$$
$$\int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Lagrange multiplier

To solve $\max f(\mathbf{x})$ s.t. $g(\mathbf{x}) \leq 0$, define $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$ and solve $\nabla \mathcal{L} = 0$ and/or $\nabla f = 0$ with $g < 0$.

2.7 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$
$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$
$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.8 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, \ (-\infty < x < \infty)$$
$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, \ (-1 < x \leq 1)$$
$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, \ (-1 \leq x \leq 1)$$
$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, \ (-\infty < x < \infty)$$
$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, \ (-\infty < x < \infty)$$

2.9 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.9.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n, p)$, $n = 1, 2, \ldots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np, \ \sigma^2 = np(1-p)$$

$\operatorname{Bin}(n, p)$ is approximately $\operatorname{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\operatorname{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1}, \ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p}, \ \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\operatorname{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots$$

$$\mu = \lambda, \ \sigma^2 = \lambda$$

2.9.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\operatorname{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\operatorname{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

Combinatorial (3)

3.1 Permutations

3.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

044568, 6 lines

```
int permToInt(vi& v) {
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & ~(1<<x)),
        use |= 1 << x; // (note: minus, not ~!)
    return r;
}
```

3.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

Similarly, let $g(n)$ be the EGF of number of graphs with n vertices satisfying some constraint, and $f(n)$ be that of connected graphs. We have $g(n) = \exp(f(n))$ or $f(n) = \ln(g(n))$.

3.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

3.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

3.2 Partitions and subsets

3.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\sim 2\text{e}5$	$\sim 2\text{e}8$

3.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

3.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$.a0a312, 6 lines

```
ll multinomial(vi& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i, 1, sz(v)) rep(j, 0, v[i])
        c = c * ++m / (j+1);
    return c;
}
```

3.3 General purpose numbers

3.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).

$$B[0, \dots] = [1, -\tfrac{1}{2}, \tfrac{1}{6}, 0, -\tfrac{1}{30}, 0, \tfrac{1}{42}, \dots]$$

Sums of powers:

$$\sum_{i=1}^n i^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^\infty f(i) = \int_m^\infty f(x) dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^\infty f(x) dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

3.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n, k) = c(n-1, k-1) + (n-1)c(n-1, k), \quad c(0, 0) = 1$$

$$\sum_{k=0}^n c(n, k) x^k = x(x+1) \dots (x+n-1)$$

$$c(8, k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$

$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

3.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n, k) = (n-k)E(n-1, k-1) + (k+1)E(n-1, k)$$

$$E(n, 0) = E(n, n-1) = 1$$

$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

3.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n-1, k-1) + kS(n-1, k)$$

$$S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

3.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. We have

$$B_n = \sum_{k=0}^{n-1} \binom{n-1}{k} B_k = \sum_{k=0}^n S(n, k)$$

For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

3.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$
with degrees d_i : $(n-2)! / ((d_1-1)! \dots (d_n-1)!)$

3.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)n!}$$
$$C_0 = 1, C_{n+1} = \frac{2(2n+1)}{n+2} C_n, C_{n+1} = \sum C_i C_{n-i}$$
$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n + 1$ leaves (0 or 2 children).
- ordered trees with $n + 1$ vertices.
- ways a convex polygon with $n + 2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

"NumberTheoreticTransform.h", "../number-theory/ModPow.h", "../number-theory/ModSqrt.h"

85a290, 188 lines

```
template <ll mod = 998244353, ll root = 3> // h: cfe604
struct Poly : vl {
    typedef Poly P;
    static int bc(int n)
    { return n <= 1 ? 1 : 1 << (32 - __builtin_clz(n - 1)); }
    static NTT<mod, root> ntt; // coefficients in [0, P)
    explicit Poly(int n = 1) : vl(n) {}
    int n() const { return (int) size(); }
    static ll modsub(ll a, ll b) { return a-b + (a<b?mod:0); }
    static ll modinv(ll x) { return modpow(x, mod - 2, mod); }
    Poly(const vl &v) : vl(v) {}
    Poly(const P &p, int m) : vl(m)
    { copy_n(p.data(), min(sz(p), m), data()); } // h: cfe604

    P &irev() { return reverse(all()), *this; } // h: a92770
    P &isz(int s) { return resize(s), *this; }
    P &iadd(const P &o) { // n() == o.n()
        rep(i,0,n()) (*this)[i] = ((*this)[i] + o[i]) % mod;
        return *this; }
    P &imul(ll k) {
        rep(i,0,n()) (*this)[i] = ((*this)[i] * k) % mod;
        return *this; }
    P &imul(P v) {
        rep(i,0,n()) (*this)[i] = ((*this)[i] * v[i]) % mod;
        return *this; }
    P &trim() {
        while (n() > 1 and back() == 0) pop_back();
        return *this; } // h: a92770

    P Mul(const P &o) const { // h: 3103e7
        const auto s = bc(n() + sz(o) - 1), inv = modinv(s);
        P x(*this, s), y(o, s), out(s);
        ntt(x.data(), s), ntt(y.data(), s);
        rep(i,0,s) out[-i & (s-1)] = x[i] * y[i] % mod * inv % mod;
        ntt(out.data(), s);
        return out.isz(n() + sz(o) - 1);
    } // h: 3103e7

    P Inv() const { assert(*begin() != 0); // h: 60c587
```

```
    if (n() == 1) return vl{modinv(*begin())};
    const auto s = bc(n() * 2), inv = modinv(s);
    P x = P(*this, (n()+1)/2).Inv().isz(s), y(*this,s), out(s);
    ntt(x.data(), s), ntt(y.data(), s);
    rep(i, 0, s) {
        ll &t = out[-i & (s - 1)];
        t = x[i] * modsub(2, x[i]*y[i]%mod) % mod * inv % mod;
    }
    ntt(out.data(), s);
    return out.isz(n());
} // h: 60c587

P sqimp() const { // coef[0] \in [1, mod)^2 h: 03c65d
    if (n() == 1) return vl{modsqrt((*this)[0], mod)};
    P x = P(*this, (n() + 1) / 2).Sqrt().isz(n());
    return x.iadd(Mul(x.Inv()).isz(n())).imul(mod / 2 + 1);
}

P Sqrt() const { // returns {-1} on fail
    int m = 0; while (m < n() and (*this)[m] == 0) ++m;
    if (m == n()) return P(n());
    if (m % 2 or modsqrt((*this)[m], mod) == -1) return vl{-1};
    P p = P(vl{data() + m, data() + n()}, n() - m/2).sqimp();
    return p.irev().isz(sz(p) + m / 2).irev();
} // h: 03c65d

pair<P, P> DivMod(P o) const { // {0} for 0 h: b479f2
    if (n() < sz(o)) return {vl{0}, *this};
    const int s = n() - sz(o) + 1;
    P x(o); x.irev().isz(s);
    P y(*this); y.irev().isz(s);
    P Q = y.Mul(x.Inv()).isz(s).irev();
    x = o.Mul(Q), y = *this;
    rep(i, 0, n()) y[i] = modsub(y[i], x[i]);
    return {Q, y.isz(max(1, sz(o)-1))};
} // h: b479f2

P Dx() const { // h: 8ed669
    P ret(n() - 1);
    rep(i, 0, sz(ret)) ret[i] = (i + 1) * (*this)[i + 1] % mod;
    return ret.isz(max(1, sz(ret)));
} // h: 8ed669
P Sx() const { // h: 1af11a
    P ret(n() + 1);
    rep(i, 0, n()) ret[i+1] = modinv(i + 1) * (*this)[i] % mod;
    return ret;
} // h: 1af11a
P Ln() const { // coef[0] == 1 h: 1bf007
    return Dx().Mul(Inv()).Sx().isz(n());
} // h: 1bf007
P Exp() const { // coef[0] == 0 h: 6c0a29
    if (n() == 1) return vl{1};
    P x = P(*this, (n() + 1) / 2).Exp().isz(n());
    P y = x.Ln(); y[0] = mod - 1;
    rep(i, 0, n()) y[i] = modsub((*this)[i], y[i]);
    return x.Mul(y).isz(n());
} // h: 6c0a29

P Pow(const string &K) const { // h: 2c36c4
    int nz = 0; ll nk = 0, nk2 = 0;
    while (nz < n()) && !(*this)[nz]) ++nz;
    for (char c : K) {
        nk = (nk * 10 + c - '0') % mod;
        nk2 = nk2 * 10 + c - '0';
        if (nk2 * nz >= n()) return P(n());
        nk2 %= mod - 1;
    }
    if (!nk && !nk2) return P(vl{1}, n());
    P x = vl(data() + nz, data() + n() - nz * (nk2 - 1));
    ll x0 = x[0];
```

```
    return x.imul(modinv(x0)).Ln().imul(nk).Exp().
        imul(modpow(x0, nk2, mod)).irev().isz(n()).irev();
} // h: 2c36c4

P tmul(int nn, const P &rhs) const { // h: 5d5fc3
    P Y = Mul(rhs).isz(n() + nn - 1);
    return P({Y.data() + n() - 1, Y.data() + Y.n()});
}

static vector<P> tree(const vl &x) {
    const int m = sz(x);
    vector<P> up(m * 2);
    rep(i, 0, m) up[m + i] = vl{(x[i] ? mod - x[i] : 0), 1};
    for (int i = m - 1; i > 0; --i)
        up[i] = up[i * 2].Mul(up[i * 2 + 1]);
    return up;
} // h: 5d5fc3
vl eval(const vl &x, const vector<P> &up) const { // h: 88b395
    const int m = sz(x); if (!m) return {};
    vector<P> dn(m * 2);
    dn[1]=P(up[1]).irev().isz(n()).Inv().irev().tmul(m,*this);
    rep(i,2,m*2) dn[i] = up[i^1].tmul(up[i].n()-1, dn[i/2]);
    vl y(m); rep(i, 0, m) y[i] = dn[m + i][0];
    return y;
} // h: 88b395
vl Eval(const vl &x) const { return eval(x, tree(x)); }
// h: 6ba1c1
static P Interpolate(const vl &x, const vl &y) { // 1e5, 1.4s
    const int m = sz(x);
    vector<P> up = tree(x), dn(m * 2);
    vl z = up[1].Dx().eval(x, up);
    rep(i, 0, m) z[i] = y[i] * modinv(z[i]) % mod;
    rep(i, 0, m) dn[m + i] = vl{z[i]};
    for (int i = m - 1; i > 0; --i)
        dn[i]=dn[i*2].Mul(up[i*2+1]).iadd(dn[i*2+1].Mul(up[i*2]));
    return dn[1];
} // h: 6ba1c1

// a..n = \sum c..j a..(n-j) h: 641b39
static ll LinearRecursion(const vl &a, const vl &c, ll n) {
    const int k = sz(a); assert(sz(c) == k + 1);
    P C(k + 1), W(vl{1}, k), M = vl{0, 1};
    rep(i, 1, k + 1) C[k - i] = (c[i] == 0 ? 0 : mod - c[i]);
    C[k] = 1;
    while (n) {
        if (n % 2) W = W.Mul(M).DivMod(C).second;
        n /= 2; M = M.Mul(M).DivMod(C).second;
    }
    ll ret = 0;
    rep(i, 0, k) ret = (ret + W[i] * a[i]) % mod;
    return ret;
} // h: 641b39

P TaylorShift(ll c) const { // returns f(x + c) h: d696a9
    P fac(n()), caf(n()); fac[0] = 1;
    rep (i, 1, n()) fac[i] = fac[i-1] * i % mod;
    rep (i, 0, n()) caf[i] = modinv(fac[i]);
    P x = P(*this).imul(fac), y(n()); ll w = 1;
    rep (i, 0, n()) y[i] = w * caf[i] % mod, w = w * c % mod;
    return x.irev().Mul(y).isz(n()).irev().imul(caf);
} // h: d696a9
// a[i]=f(i), i in [0, n) => a[i]=f(c + i), i in [0, m)
P SamplingShift(int m, ll c) const { // h: 9d4306
    const int k = max(n(), m);
    P fac(k), caf(k); fac[0] = 1;
    rep (i, 1, k) fac[i] = fac[i-1] * i % mod;
    rep (i, 0, k) caf[i] = modinv(fac[i]);
    P x = P(*this).imul(caf), y = caf;
    rep (i, 0, n()) if (i & 1)
```

```

    y[i] = (y[i] == 0 ? 0 : mod - y[i]);
    x = x.Mul(y).isz(n()).imul(fac).irev();
    ll w = 1;
    rep(i, 0, n()) y[i] = caf[i] * w % mod,
        w = w * modsub(c, i) % mod;
    x = x.Mul(y).isz(n()).irev().imul(caf).isz(m);
    y = caf; y = y.isz(m);
    return x.Mul(y).isz(m).imul(fac);
} // h: 9d4306
};
using Poly_t = Poly<998244353, 62>;
template<> decltype(Poly_t::ntt) Poly_t::ntt = {};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.

Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve x²-3x+2 = 0

Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

"Polynomial.h"ad5c22, 23 lines

```
vector<double> polyRoots(Poly<> p, double xmin, double xmax) {
    if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
    vector<double> ret;
    Poly<> der = p;
    der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.push_back(xmin-1);
    dr.push_back(xmax+1);
    sort(all(dr));
    rep(i,0,sz(dr)-1) {
        double l = dr[i], h = dr[i+1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            rep(it,0,60) { // while (h - l > 1e-8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign) l = m;
                else h = m;
            }
            ret.push_back((l + h) / 2);
        }
    }
    return ret;
}
```

PolyInterpolate.h

Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.

Time: $\mathcal{O}(n^2)$

08bf48, 13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    rep(k,0,n-1) rep(i,k+1,n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k,0,n) rep(i,0,n) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.

Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}

Time: $\mathcal{O}(N^2)$

"../number-theory/ModPow.h"3a4764, 21 lines

```
const ll mod = 5;
vector<ll> berlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i,0,n) { ++m;
        ll d = s[i] % mod;
        rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * modpow(b, mod-2, mod) % mod;
        rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    for (ll& x : C) x = (mod - x) % mod;
    return C;
}
```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i - j - 1]tr[j]$, given $S[0 \dots \geq n - 1]$ and $tr[0 \dots n - 1]$. Faster than matrix multiplication. Useful together with Berlekamp-Massey.

Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number

Time: $\mathcal{O}(n^2 \log k)$

f4e444, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        rep(i,0,n+1) rep(j,0,n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep(j,0,n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is eps . Works equally well for maximization with a small change in the code. See Ternary-Search.h in the Various chapter for a discrete version.

Usage: double func(double x) { return 4+x+.3*x*x; }
double xmin = gss(-1000,1000,func);

Time: $\mathcal{O}(\log((b - a)/\epsilon))$

d7b114, 15 lines

template <class F>

```
double gss(double a, double b, F f) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}
```

HillClimbing.h

Description: Poor man's optimization for unimodal functions

Seeef, 14 lines

```
typedef array<double, 2> P;

template<class F> pair<double, P> hillClimb(P start, F f) {
    pair<double, P> cur(f(start), start);
    for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
        rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
            P p = cur.second;
            p[0] += dx*jmp;
            p[1] += dy*jmp;
            cur = min(cur, make_pair(f(p), p));
        }
    }
    return cur;
}
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

4756fc, 7 lines

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
    double h = (b - a) / 2 / n, v = f(a) + f(b);
    rep(i,1,n*2)
        v += f(a + i*h) * (i&1 ? 4 : 2);
    return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.

Usage: double sphereVolume = quad(-1, 1, [](double x) { return quad(-1, 1, [&](double y) { return quad(-1, 1, [&](double z) { return x*x + y*y + z*z < 1; }}});});

92dd79, 15 lines

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
    d c = (a + b) / 2;
    d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
    if (abs(T - S) <= 15 * eps || b - a < 1e-10)
        return T + (T - S) / 15;
    return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}

template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
    return rec(f, a, b, eps, S(a, b));
}
```

```
}

Simplex.h
Description: Solves a general linear maximization problem: maximize  $c^T x$ 
subject to  $Ax \leq b, x \geq 0$ . Returns -inf if there is no solution, inf if there
are arbitrarily good solutions, or the maximum value of  $c^T x$  otherwise. The
input vector is set to an optimal  $x$  (or in the unbounded case, an arbitrary
solution fulfilling the constraints). Numerical stability is not guaranteed. For
better performance, define variables such that  $x = 0$  is viable.
Usage: simplex.init(n, m);
simplex.a[i][j] = aij;
int val = simplex.solve();
Time:  $\mathcal{O}(NM * \#pivots)$ , where a pivot may be e.g. an edge relaxation.
 $\mathcal{O}(2^N)$  in the general case.
cc78c7, 66 lines
```

```
static constexpr long double eps = 1e-7;
template <class T = double>
struct Simplex {
    int n, m;
    vector<int> l, d;
    vector<vector<T>> a;
    vector<T> b, c, sol;
    T v;

    bool eq(T a, T b) { return fabs(a - b) < eps; }
    bool ls(T a, T b) { return a < b && !eq(a, b); }

    void init(int p, int q) {
        n = p; m = q; v = 0;
        l.assign(m, 0); b = l;
        d.assign(n, 0); c = sol = d;
        a.assign(m, vector<T>(n, 0));
    }

    void pivot(int x,int y) {
        swap(l[x], d[y]);
        T k = a[x][y]; a[x][y] = 1;
        vector<int> nz;
        rep (i, 0, n) {
            a[x][i] /= k;
            if(!eq(a[x][i], 0)) nz.push_back(i);
        }
        b[x] /= k;
        rep (i, 0, m) {
            if(i == x || eq(a[i][y], 0)) continue;
            k = a[i][y]; a[i][y] = 0;
            b[i] -= k*b[x];
            for(int j : nz) a[i][j] -= k * a[x][j];
        }
        if(eq(c[y], 0)) return;
        k = c[y]; c[y] = 0;
        v += k * b[x];
        for(int i : nz) c[i] -= k * a[x][i];
    }

    // 0: found solution, 1: no feasible solution, 2: unbounded
    int solve() {
        rep (i, 0, n) d[i] = i;
        rep (i, 0, m) l[i] = n+i;
        while(1) { // Eliminating negative b[i]
            int x = -1, y = -1;
            rep (i, 0, m) if (ls(b[i], 0) && (x == -1 || b[i] < b[x]))
                x = i;
            if(x == -1) break;
            rep (i, 0, n) if (ls(a[x][i], 0) && (y == -1 || a[x][i] <
                a[x][y])) y = i;
            if(y == -1) return 1;
            pivot(x, y);
        }
    }
};
```

```
while(1) {
    int x = -1, y = -1;
    rep (i, 0, n)
        if (ls(0, c[i]) && (y == -1 || c[i] > c[y])) y = i;
    if(y == -1) break;
    rep (i, 0, m)
        if (ls(0, a[i][y]) && (x == -1 || b[i]/a[i][y] < b[x]/a
            [x][y])) x = i;
    if(x == -1) return 2;
    pivot(x, y);
}
rep (i, 0, m) if(l[i] < n) sol[l[i]] = b[i];
return 0;
};
```

Duality.h

Description: Finds the Dual problem of an LP Maximize $Z = c^T x \leftrightarrow$ Minimize $W = y^T b$ s.t. $Ax \leq b \leftrightarrow$ s.t. $A^T y \geq c$ and $x \geq 0 \leftrightarrow$ and $y \geq 0$. variables to constraints, constraints to variables. weak duality property: any feasible solution x of a primal problem and any feasible solution y of the dual problem dual problem satisfies $c^T x \leq b^T y$. Strong duality property: any feasible solution x of a primal problem and any feasible solution y of the dual problem dual problem satisfies $c^T x = b^T y$.

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Time: $\mathcal{O}(N^3)$

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.

Time: $\mathcal{O}(N^3)$

3313dc, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
    int n = sz(a); ll ans = 1;
    rep(i,0,n) {
        rep(j,i+1,n) {
            while (a[j][i] != 0) { // gcd step
                ll t = a[i][i] / a[j][i];
                if (t) rep(k,i,n)
                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }
}
```

```
return (ans + mod) % mod;
}

SolveLinear.h
Description: Solves  $A * (x + B^T y) = b$ , where  $y$  is an arbitrary vector. If
there are multiple solutions, an arbitrary one is returned. Returns rank, or
-1 if no solutions. B are the m - rank solutions for  $Ax = 0$ .
Time:  $\mathcal{O}(n^2 m)$ 
335564, 48 lines

const double eps = 1e-12;
template <class T>
struct LinearSolver {
    typedef vector<T> vt;
    vector<vt> A, B;
    vt b, x;
    int rank;
    LinearSolver(const vector<vt>& _A, const vt& _b) :
        A(_A), b(_b), rank(0) {
        int n = sz(A), m = sz(A[0]), br, bc;
        vi col(m); iota(all(col), 0);

        rep (i, 0, n) {
            T v, bv = T(0);
            rep (r, i, n) rep(c, i, m)
                if ((v = fabs(A[r][c])) > bv) // mod: v != 0
                    br = r, bc = c, bv = v;
            if (bv <= eps) { // mod: bv == 0 / b[j] != 0
                rep (j, i, n) if (fabs(b[j]) > eps) {
                    rank = -1; return; }
                break;
            }
            swap(A[i], A[br]);
            swap(b[i], b[br]);
            swap(col[i], col[bc]);
            rep (j, 0, n) swap(A[j][i], A[j][bc]);
            rep (j, i+1, m) A[i][j] /= A[i][i];
            b[i] /= A[i][i];
            A[i][i] = T(1);
            rep (j, 0, n) if (j != i) {
                T fac = A[j][i];
                b[j] -= fac * b[i];
                rep (k, i, m) A[j][k] -= fac * A[i][k];
            }
            rank++;
        }
        x.assign(m, T(0));
        for (int i = rank; i--;) {
            x[col[i]] = b[i];
            rep (j, 0, i) b[j] -= A[j][i] * b[i];
        }
        rep (i, rank, m) { // homo sols
            vt sol(m, T(0)); sol[col[i]] = T(1);
            rep (j, 0, rank) sol[col[j]] = (A[j][i] * T(-1));
            B.push_back(sol);
        }
    }
};
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .

Time: $\mathcal{O}(n^2 m)$

fa2d7a, 34 lines

```
typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vi col(m); iota(all(col), 0);
```

```
rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
        rep(j,i,n) if(b[j]) return -1;
        break;
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
        A[j].flip(i); A[j].flip(bc);
    }
    rep(j,i+1,n) if (A[j][i]) {
        b[j] ^= b[i];
        A[j] ^= A[i];
    }
    rank++;
}

x = bs();
for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
}
return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: Invert matrix A modulo a prime. Returns rank; result is stored in A unless singular (rank < n). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p , and k is doubled in each step.

Time: $\mathcal{O}(n^3)$

"/..number-theory/ModPow.h" f61174, 39 lines

```
const ll mod = 998244353;
template <class T>
int matInv(vector<vector<T>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<T>> tmp(n, vector<T>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;

    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n)
            if (A[j][k]) // fp: fabs(A[j][k]) > fabs(A[r][c])
                r = j, c = k;
        if (A[r][c]) // fp: return if fabs(A[r][c]) <= eps
            return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        T v = modpow(A[i][i], mod - 2, mod);
        rep(j,i+1,n) {
            T f = A[j][i] * v % mod;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
            rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) % mod;
        }
        rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
        rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
        A[i][i] = 1;
    }
}
```

```
for (int i = n-1; i > 0; --i) rep(j,0,i) {
    T v = A[j][i];
```

```
        rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
    }

    rep(i,0,n) rep(j,0,n)
        A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*mod;
    return n;
}
```

CharPoly.h

Description: Calculate $\det(xI - M)$.

Time: $\mathcal{O}(N^3)$

"/..number-theory/ModPow.h" e6af4a, 34 lines

```
const ll mod = 998244353;
template <class T>
void Hessenberg(int n, vector<vector<T>> &a) {
    rep (i, 0, n - 2) {
        rep (j, i + 1, n) if (a[j][i]) {
            rep (k, i, n) swap(a[i+1][k], a[j][k]);
            rep (k, 0, n) swap(a[k][i+1], a[k][j]);
            break;
        }
        if (a[i + 1][i] == 0) continue;
        rep (j, i + 2, n) {
            T c = modpow(a[i + 1][i], mod-2, mod) * a[j][i] % mod;
            rep (k, i, n)
                a[j][k] = (a[j][k] - a[i+1][k] * c % mod + mod) % mod;
            rep (k, 0, n) a[k][i+1] = (a[k][i+1] + a[k][j] * c) % mod;
        }
    }
}

template <class T>
vector<T> CharPoly(vector<vector<T>> a) {
    int n = sz(a); Hessenberg(n, a);
    vector<vector<T>> p(n + 1, vector<T>(n + 1)); p[0][0] = 1;
    rep (i, 1, n + 1) {
        rep (j, 0, i + 1) p[i][j] = j ? p[i - 1][j - 1] : 0;
        T val = 1;
        for (int j = i - 1; j >= 0; --j) {
            T c = val * a[j][i - 1] % mod;
            rep (k, 0, j + 1)
                p[i][k] = (p[i][k] - p[j][k] * c % mod + mod) % mod;
            if (j) val = val * a[j][j - 1] % mod;
        }
    }
    return p[n];
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither tr nor the check for $\text{diag}[i] == 0$ is needed.

Time: $\mathcal{O}(N)$

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        } else {
            b[i] /= diag[i];
            if (i) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: $\text{fft}(a)$ computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: $\text{conv}(a, b) = c$, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , $\text{reverse}(\text{start}+1, \text{end})$, FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTmod.

Time: $\mathcal{O}(n \log n)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}

vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
```

```
rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
fft(out);
rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N\log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h"b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i]b[x - i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h"786630, 37 lines

```
typedef vector<ll> vl;
template <ll mod, ll root>
struct NTT {
    void operator()(ll a[], int n) {
        int L = 31 - __builtin_clz(n);
        static vl rt(2, 1);
        for (static int k = 2, s = 2; k < n; k *= 2, s++) {
            rt.resize(n);
            ll z[] = {1, modpow(root, mod >> s, mod)};
            rep(i, k, 2 * k) rt[i] = rt[i / 2] * z[i & 1] % mod;
        }
        vi rev(n);
        rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
        rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
        for (int k = 1; k < n; k *= 2)
            for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
                ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
                a[i + j + k] = ai - z + (z > ai ? mod : 0);
                ai += (ai + z >= mod ? z - mod : z);
            }
    }
};
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
```

```
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
NTT<mod, root> ntt;
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1 << (32 - __builtin_clz(s));
    ll inv = modpow(n, mod - 2, mod);
    vl x(a), y(b), out(n);
    x.resize(n); y.resize(n);
    ntt(x.data(), n); ntt(y.data(), n);
    rep(i,0,n) out[-i & (n-1)] = x[i] * y[i] % mod * inv % mod;
    ntt(out.data(), n);
    return {begin(out), begin(out) + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

464cf3, 16 lines

```
void FST(vi& a, bool inv) { // remember to take mod
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
                inv ? pii(v - u, u) : pii(v, u + v); // AND
                inv ? pii(v, u - v) : pii(u + v, u); // OR
                pii(u + v, u - v); // XOR
        }
        if (inv) for (int& x : a) x /= sz(a); // XOR only
    }
}
vi conv(vi a, vi b) {
    FST(a, 0); FST(b, 0);
    rep(i,0,sz(a)) a[i] *= b[i];
    FST(a, 1); return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$

c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i,2,n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.

$\text{modsum}(\text{to}, c, k, m) = \sum_{i=0}^{\text{to}-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.

5c5bc5, 16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \pmod c$ (or $a^b \pmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

bbbd8f, 11 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h"f164c2, 24 lines

```
ll modsqrt(ll a, ll p) { // return -1 on fail
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    if (modpow(a, (p-1)/2, p) != 1) return -1;
    if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for (; r = m) {
        ll t = b;
        for (m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if (m == 0) return min(x, p - x);
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.

Time: LIM=1e9 $\approx 1.5s$

6b2912, 20 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
```

```
const int S = (int)round(sqrt(LIM)), R = LIM / 2;
vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
vector<pii> cp;
for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
}
for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
        for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i,0,min(S, R - L))
        if (!block[i]) pr.push_back((L + i) * 2 + 1);
}
for (int i : pr) isPrime[i] = 1;
return pr;
}
```

MillerRabin.h
Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$.

```
"ModMulLL.h" 60dcd1, 12 lines

bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}
```

Factor.h
Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

```
"ModMulLL.h", "MillerRabin.h" d8d98d, 18 lines

ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

5.3 Divisibility

Euclid.h
Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

CRT.h
Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

```
"Euclid.h" 04d93a, 7 lines

ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

5.3.1 Bézout’s identity

For $a \neq 0, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h
Description: *Euler’s* ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p \text{ prime} \Rightarrow \phi(p^k) = (p - 1)p^{k-1}, m, n \text{ coprime} \Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.
 $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler’s thm: $a, n \text{ coprime} \Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat’s little thm: $p \text{ prime} \Rightarrow a^{p-1} \equiv 1 \pmod p \ \forall a$.

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

ContinuedFractions.h
Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.
Time: $\mathcal{O}(\log N)$

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
    for (;;) {
        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
```

```
        NP = b*P + LP, NQ = b*Q + LQ;
        if (a > b) {
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if (abs(y = 1/(y - (d)a)) > 3*(d)N) {
            return {NP, NQ};
        }
        LP = P; P = NP;
        LQ = Q; Q = NQ;
    }
}
```

FracBinarySearch.h
Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: `fracBS([](Frac f) { return f.p>=3*f.q; }, 10);` // {1,3}
Time: $\mathcal{O}(\log(N))$

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if (f(lo)) return lo;
    assert(f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step; si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B; B = !adv;
    }
    return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$. Highly composite numbers:

<i>n</i>	840	7560	83160	720720	8648640	735134400	963761198400
<i>σ</i> (<i>n</i>)	32	64	128	240	448	1344	6720

(866421317361600, 26880), (897612484786617600, 103680)

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

Geometry (6)

6.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 29 lines

```
// for double: (x > eps) - (x < -eps)
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()==1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << ", " << p.y << ")"; }
};
```

LineDistance.h

Description: Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

f6bf6b, 3 lines

```
template<class P>
double lineDist(const P& a, const P& b, const P& p)
{ return (double) (b-a).cross(p-a)/(b-a).dist(); }
```

LineIntersection.h

Description: If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d}; // C^3
}
```

SegmentDistance.h

Description: Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1); bool onSegment = segDist(a,b,p) < 1e-10;

7b886d, 6 lines

```
template <class P>
double segDist(P& s, P& e, P& p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d; // C^3
}
```

SegmentIntersection.h

Description: If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2); if (sz(inter)==1) cout << "segments intersect at " << inter[0] << endl;

9d5f72, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
```

SideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;

3af81c, 9 lines

```
oc = a.cross(b, c), od = a.cross(b, d);
// Checks if intersection is single non-endpoint point.
if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)}; // C^3
set<P> s;
if (onSegment(c, d, a)) s.insert(a);
if (onSegment(c, d, b)) s.insert(b);
if (onSegment(a, b, c)) s.insert(c);
if (onSegment(a, b, d)) s.insert(d);
return {all(s)};
}
```

SideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;

3af81c, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

1ee809, 2 lines

```
template<class P> bool onSegment(P s, P e, P p)
{ return sgn(p.cross(s, e)) == 0 && sgn((s-p).dot(e-p)) <= 0; }
```

LinearTransformation.h

Description: Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

03a306, 6 lines

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

b5562d, 5 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
    P v = b - a;
    return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted

```
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively
oriented triangles with vertices at 0 and i
0f0602, 35 lines
```

struct Angle {
 int x, y;
 int t;
 Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
 Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
 int half() const {
 assert(x || y);
 return y < 0 || (y == 0 && x < 0);
 }
 Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
 Angle t180() const { return {-x, -y, t + half()}; }
 Angle t360() const { return {x, y, t + 1}; }
};

bool operator<(Angle a, Angle b) {
 // add a.dist2() and b.dist2() to also compare distances
 return make_tuple(a.t, a.half(), a.y * (1l)b.x) <
 make_tuple(b.t, b.half(), a.x * (1l)b.y);
}

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
 if (b < a) swap(a, b);
 return (b < a.t180() ?
 make_pair(a, b) : make_pair(b, a.t360()));
}

Angle operator+(Angle a, Angle b) { // point a + vector b
 Angle r(a.x + b.x, a.y + b.y, a.t);
 if (a.t180() < r) r.t--;
 return r.t180() < a ? r.t360() : r;
}

Angle angleDiff(Angle a, Angle b) { // angle b - angle a
 int tu = b.t - a.t; a.t = b.t;
 return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}

6.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h"70ff65, 10 lines

typedef Point<double> P;
vector<P> circleInter(P a,P b,double r1,double r2) {
 if (a == b) { assert(r1 != r2); return {}; }
 P vec = b - a;
 double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
 p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
 if (sum*sum < d2 || dif*dif > d2) return {};
 P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
 return {mid + per, mid - per};
}

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents - 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h"b2c9e1, 13 lines

typedef Point<double> P;
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {

```
P d = c2 - c1;
double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
if (d2 == 0 || h2 < 0) return {};
vector<pair<P, P>> out;
for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
}
if (h2 == 0) out.pop_back();
return out;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h"df13e1, 9 lines

typedef Point<double> P;
vector<P> circleLine(P c, double r, P a, P b) {
 P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
 double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
 if (h2 < 0) return {};
 if (h2 == 0) return {p};
 P h = ab.unit() * sqrt(h2);
 return {p - h, p + h};
}

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

".../content/geometry/Point.h"a1ee63, 19 lines

typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
 auto tri = [&](P p, P q) {
 auto r2 = r * r / 2;
 P d = q - p;
 auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
 auto det = a * a - b;
 if (det <= 0) return arg(p, q) * r2;
 auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
 if (t < 0 || 1 <= s) return arg(p, q) * r2;
 P u = p + d * s, v = p + d * t;
 return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
 };
 auto sum = 0.0;
 rep(i,0,sz(ps))
 sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
 return sum;
}

Circumcircle.h

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.

"Point.h"940f8d, 20 lines

template <class P> // in/on/out: >=/< 0 h: fef362
int inCircumcircle(P o, P a, P b, P c) {
 a = a - o, b = b - o, c = c - o;
 if (a.cross(b, c) < 0) swap(b, c);
 __int128 za = a.dist2(), zb = b.dist2(), zc = c.dist2();
 __int128 t = za * b.cross(c) -
 zb * a.cross(c) + zc * a.cross(b); // C^4
 return sgn(t);
}

```
} // h: fef362

template <class P>
double ccRadius(const P& A, const P& B, const P& C) {
    return (B-A).dist()*(C-B).dist()*(A-C).dist() /
        abs((B-A).cross(C-A))/2;
}

template <class P>
P ccCenter(const P& A, const P& B, const P& C) {
    P b = C-A, c = B-A;
    return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

"Circumcircle.h"ff0215, 22 lines

template <class P>
vector<P> mec(vector<P> ps) {
 sort(all(ps)); ps.erase(unique(all(ps)), end(ps));
 shuffle(all(ps), mt19937(time(0)));
 vector<P> circ(1, ps[0]);
 const auto out = [&](P p) {
 if (sz(circ) == 1) return not (p == circ[0]);
 if (sz(circ) == 2)
 return sgn((p * 2 - (circ[0] + circ[1])).dist2()
 - (circ[0] - circ[1]).dist2()) > 0;
 return inCircumcircle(p, circ[0], circ[1], circ[2]) < 0;
 };
 rep(i,0,sz(ps)) if (out(ps[i])) {
 circ = {ps[i]};
 rep(j,0,i) if (out(ps[j])) {
 circ = {ps[i], ps[j]};
 rep(k,0,j) if (out(ps[k]))
 circ = {ps[i], ps[j], ps[k]};
 }
 }
 return circ;
}

6.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};

bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"2bf504, 11 lines

template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
 int cnt = 0, n = sz(p);
 rep(i,0,n) {
 P q = p[(i + 1) % n];
 if (onSegment(p[i], q, a)) return !strict;
 //or: if (segDist(p[i], q, a) <= eps) return !strict;
 cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
 }
 return cnt;
}

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"536a15, 6 lines

template<class T>
T polygonArea2(const vector<Point<T>>& v) {

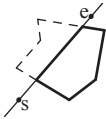
```
T a = v.back().cross(v[0]);
rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
return a;
}
```

PolygonCenter.h
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$

"Point.h"	9706dc, 9 lines
-----------	-----------------

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

PolygonCut.h
Description:
Returns a vector with the vertices of a polygon with every-thing to the left of the line going from s to e cut away.
Usage: vector<P> p = ...;
p = polygonCut(p, P(0,0), P(1,0));



"Point.h", "LineIntersection.h"	f2b7d4, 13 lines
---------------------------------	------------------

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
    vector<P> res;
    rep(i,0,sz(poly)) {
        P cur = poly[i], prev = i ? poly[i-1] : poly.back();
        bool side = s.cross(e, cur) < 0;
        if (side != (s.cross(e, prev) < 0))
            res.push_back(lineInter(s, e, cur, prev).second);
        if (side)
            res.push_back(cur);
    }
    return res;
}
```

PolygonUnion.h
Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Time: $\mathcal{O}(N^2)$, where N is the total number of points

"Point.h", "SideOf.h"	3931c6, 33 lines
-----------------------	------------------

```
typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
    double ret = 0;
    rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
        P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
        vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
        rep(j,0,sz(poly)) if (i != j) {
            rep(u,0,sz(poly[j])) {
                P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
                int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
                if (sc != sd) {
                    double sa = C.cross(D, A), sb = C.cross(D, B);
                    if (min(sc, sd) < 0)
                        segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
                } else if (!sc && !sd && j < i && sgn((B-A).dot(D-C)) > 0) {
                    segs.emplace_back(rat(C - A, B - A), 1);
                    segs.emplace_back(rat(D - A, B - A), -1);
                }
            }
        }
    }
```

```
}
sort(all(segs));
for (auto& s : segs) s.first = min(max(s.first, 0.0), 1.0);
double sum = 0;
int cnt = segs[0].second;
rep(j,1,sz(segs)) {
    if (!cnt) sum += segs[j].first - segs[j - 1].first;
    cnt += segs[j].second;
}
ret += A.cross(B) * sum;
}
return ret / 2;
}
```

6.4 Convex Hull

ConvexHull.h
Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.
Time: $\mathcal{O}(n \log n)$



"Point.h"	b50acd, 13 lines
-----------	------------------

```
template <class P>
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<P> h(sz(pts)+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h
Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

"Point.h"	10aada, 11 lines
-----------	------------------

```
template <class P>
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i,0,j) for (; j = (j + 1) % n) {
        res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
        if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
            break;
    }
    return res.second;
}
```

PointInsideHull.h
Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

"Point.h", "SideOf.h", "OnSegment.h"	71446b, 14 lines
--------------------------------------	------------------

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
        return false;
```

```
while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
}
return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h
Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"Point.h"	7cf45b, 39 lines
-----------	------------------

```
#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
    }
    return lo;
}
```

```
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i,0,2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
            case 0: return {res[0], res[0]};
            case 2: return {res[1], res[1]};
        }
    return res;
}
```

MinkowskiSum.h
Description: Output the minkowski sum for two convex hull. For distance of two convex hull A and B, calculate distance from O(0, 0) to minkowskiSum(A, -B).

"Point.h"	ba4047, 16 lines
-----------	------------------

```
template<class P>
vector<P> minkowskiSum(vector<P> a, vector<P> b) {
    int n = sz(a), m = sz(b);
    rotate(begin(a), min_element(all(a)), end(a));
    rotate(begin(b), min_element(all(b)), end(b));
```

```

a.push_back(a[0]); a.push_back(a[1]);
b.push_back(b[0]); b.push_back(b[1]);
vector<P> res;
for (int i = 0, j = 0; i < n or j < m; ) {
    res.push_back(a[i] + b[j]);
    auto c = sgn((a[i+1] - a[i]).cross(b[j+1] - b[j]));
    i += (i < n and c >= 0);
    j += (j < m and c <= 0);
}
return res;
}

```

HalfPlaneInter.h

Description: Given n segments (s, t), returns polygon sides representing the intersection of left side of the segments.

"Point.h"	bc38c7, 35 lines
-----------	------------------

```

template <class P> struct Seg { P s, t; };
template <class S>
bool xleft(const S& o, const S& a, const S& b) {
    auto [o3, o4] = make_pair(o.s.cross(o.t, b.s),
        o.s.cross(o.t, b.t)); // C^2
    auto [a3, a4] = make_pair(a.s.cross(a.t, b.s),
        a.s.cross(a.t, b.t));
    if (a3 - a4 < 0) a3 *= -1, a4 *= -1;
    return (__int128) o4 * a3 - (__int128) o3 * a4 > 0; // C^4
}
template <class P>
int cmp(const P& a, const P& b, const bool same = true) {
    int na = (a < P(0, 0)), nb = (b < P(0, 0));
    if (na != nb) return na < nb;
    if (sgn(a.cross(b)) != 0) return sgn(a.cross(b)) > 0;
    return same ? a.dist2() < b.dist2() : -1;
}
template<class S>
vector<S> halfPlaneInter(vector<S> ss) {
    sort(all(ss), [&](S a, S b) -> int {
        int t = cmp(a.t - a.s, b.t - b.s, 0);
        return (t != -1 ? t : sgn(a.s.cross(a.t, b.s)) < 0);
    });
    int n = sz(ss), qh = 0, qt = 1;
    vector<S> dq(n); dq[0] = ss[0];
    rep(i, 1, n) {
        if ((ss[i-1].t - ss[i-1].s).cross(ss[i].t - ss[i].s) == 0)
            continue;
        while (qt-qh>1 and !xleft(ss[i], dq[qt-2], dq[qt-1])) --qt;
        while (qt-qh>1 and !xleft(ss[i], dq[qh], dq[qh+1])) ++qh;
        dq[qt++] = ss[i];
    }
    while (qt-qh>2 and !xleft(dq[qh], dq[qt-2], dq[qt-1])) --qt;
    return {begin(dq) + qh, begin(dq) + qt};
}

```

6.5 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

"Point.h"	ac41a6, 17 lines
-----------	------------------

```

typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (ll)sqrt(ret.first), 0};
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);

```

```

        for (; lo != hi; ++lo)
            ret = min(ret, {(lo - p).dist2(), {lo, p}});
        S.insert(p);
    }
    return ret.second;
}

```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights $w(p, q) = |p.x - q.x| + |p.y - q.y|$. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.

Time: $\mathcal{O}(N \log N)$

"Point.h"	df6f59, 23 lines
-----------	------------------

```

typedef Point<int> P;
vector<array<int, 3>> manhattanMST(vector<P> ps) {
    vi id(sz(ps));
    iota(all(id), 0);
    vector<array<int, 3>> edges;
    rep(k, 0, 4) {
        sort(all(id), [&](int i, int j) {
            return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
        map<int, int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
                int j = it->second;
                P d = ps[i] - ps[j];
                if (d.y > d.x) break;
                edges.push_back({d.y + d.x, i, j});
            }
            sweep[-ps[i].y] = i;
        }
        for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
    }
    return edges;
}

```

KDTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h"	bac5b0, 63 lines
-----------	------------------

```

typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

```

```

bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }

```

```

struct Node {
    P pt; // if this is a leaf, the single point in it
    T x0 = -INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    Node *first = 0, *second = 0;

    T distance(const P& p) { // min squared distance to a point
        T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
        T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
        return (P(x,y) - p).dist2();
    }
}

```

```

Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
        x0 = min(x0, p.x); x1 = max(x1, p.x);
        y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
        // split on x if width >= height (not ideal...)
        sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
        // divide by taking half the array for each child (not

```

```

        // best performance with many duplicates in the middle)
        int half = sz(vp)/2;
        first = new Node({vp.begin(), vp.begin() + half});
        second = new Node({vp.begin() + half, vp.end()});
    }
}
};

```

```

struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}
}

```

```

pair<T, P> search(Node *node, const P& p) {
    if (!node->first) {
        // uncomment if we should not find the point itself:
        // if (p == node->pt) return {INF, P()};
        return make_pair((p - node->pt).dist2(), node->pt);
    }
}

```

```

Node *f = node->first, *s = node->second;
T bfirst = f->distance(p), bsec = s->distance(p);
if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

```

```

// search closest side first, other side if needed
auto best = search(f, p);
if (bsec < best.first)
    best = min(best, search(s, p));
return best;
}

```

```

// find nearest point to a point, and its squared distance
// (requires an arbitrary operator< for Point)
pair<T, P> nearest(const P& p) {
    return search(root, p);
}
};

```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

"Point.h"	eefdf5, 88 lines
-----------	------------------

```

typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t ll1; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point

```

```

struct Quad {
    Q rot, o; P p = arb; bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;

```

```

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    ll1 p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}

```

```

Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad(new Quad{new Quad{new Quad{0}}});
    H = r->o; r->r()->r() = r;
    rep(i, 0, 4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}

```

```
}
void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q,Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
    Q A, B, ra, rb;
    int half = sz(s) / 2;
    tie(ra, A) = rec({all(s) - half});
    tie(B, rb) = rec({sz(s) - half + all(s)});
    while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
           (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
    Q base = connect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
        Q t = e->dir; \
        splice(e, e->prev()); \
        splice(e->r(), e->r()->prev()); \
        e->o = H; H = e; e = t; \
    }
    for (;;) {
        DEL(LC, base->r(), o); DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
    sort(all(pts)); assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD; pts.clear();
    while (qi < sz(q)) if (!(e = q[qi+])->mark) ADD;
    return pts;
}
```

6.5.1 Voronoi Diagram

Perform half-plane intersection on bisectors of triangles from FastDelaunay. Deal with cases that all points are collinear (no triangles) carefully.

6.6 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6 lines

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst) {
    double v = 0;
    for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

8058ae, 32 lines

```
template<class T> struct Point3D {
    typedef Point3D P;
    typedef const P& R;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
    bool operator<(R p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(R p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
    P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
    P operator*(T d) const { return P(x*d, y*d, z*d); }
    P operator/(T d) const { return P(x/d, y/d, z/d); }
    T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
    P cross(R p) const {
        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
    }
    T dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt((double)dist2()); }
    //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
    double phi() const { return atan2(y, x); }
    //Zenith angle (latitude) to the z-axis in interval [0, pi]
    double theta() const { return atan2(sqrt(x*x+y*y),z); }
    P unit() const { return *this/(T)dist(); } //makes dist()==1
    //returns unit vector normal to *this and p
    P normal(P p) const { return cross(p).unit(); }
    //returns point rotated 'angle' radians ccw around axis
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle); P u = axis.unit();
        return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
    }
};
```

3DHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}(n^2)$

"Point3D.h"

5b45fe, 49 lines

```
typedef Point3D<double> P3;

struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
```

```
};

struct F { P3 q; int a, b, c; };

vector<F> hull3d(const vector<P3>& A) {
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if (q.dot(A[l]) > q.dot(A[i]))
            q = q * -1;
        F f{q, i, j, k};
        E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
        FS.push_back(f);
    };
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
        mf(i, j, k, 6 - i - j - k);

    rep(i,4,sz(A)) {
        rep(j,0,sz(FS)) {
            F f = FS[j];
            if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
                E(a,b).rem(f.c);
                E(a,c).rem(f.b);
                E(b,c).rem(f.a);
                swap(FS[j-], FS.back());
                FS.pop_back();
            }
        }
        int nw = sz(FS);
        rep(j,0,nw) {
            F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
            C(a, b, c); C(a, c, b); C(b, c, a);
        }
        for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
            A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
        return FS;
    };
};
```

SphericalDistance.h

Description: Returns the shortest distance on the sphere with radius r adius between the points with azimuthal angles (longitude) f_1 (ϕ_1) and f_2 (ϕ_2) from x axis and zenith angles (latitude) t_1 (θ_1) and t_2 (θ_2) from z axis ($0 =$ north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. $dx*$ radius is then the difference between the two points in the x direction and $d*$ radius is the total distance between the points.

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}
```

Strings (7)

KMP.h

Description: $p[x] = \max j$ s.t. $j \leq i$ and $s[0, j-1] = s[i-j+1, i]$, $p[0] = 0$
Time: $\mathcal{O}(|P| + |T|)$

6894ac, 19 lines

```
vi fail_function(const string &s) { // h: 5b59d3
    vi p(sz(s));
    rep (i, 1, sz(s)) {
        int j = p[i - 1];
        while (j > 0 and s[j] != s[i]) j = p[j-1];
        p[i] = j + (s[i] == s[j]);
    }
    return p;
} // h: 5b59d3
int match(const string &T, const string &P) {
    auto a = fail_function(P + '$');
    int ans = 0, j = 0;
    rep (i, 0, sz(T)) {
        while (j > 0 and T[i] != P[j]) j = a[j-1];
        j += (T[i] == P[j]);
        ans += (j == sz(P));
    }
    return ans;
}
```

ZFunc.h

Description: $z[i] = \max j$ s.t. $s[0, j) = s[i, i+j)$, $z[0] = 0$
Time: $\mathcal{O}(|P| + |T|)$

08a457, 17 lines

```
vi z_func(const string &s) { // h: c6b698
    vi z(sz(s));
    int l = 0, r = 0;
    rep (i, 1, sz(s)) {
        if (i < r) z[i] = min(r - i, z[i - 1]);
        while (i + z[i] < sz(s) and s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
} // h: c6b698
int match(const string &T, const string &P) {
    auto z = z_func(P + '$' + T);
    int ans = 0;
    rep (i, 0, sz(T))
        ans += (z[sz(P) + 1 + i] == sz(P));
    return ans;
}
```

Manacher.h

Description: Computes the longest palindromic subsequence in string. even indices are padded with char '.'.
Usage: $p.i-1 = \text{length centered at } i$
Time: $\mathcal{O}(N)$

d46d3a, 13 lines

```
vi Manacher(string s) {
    string t(sz(s) * 2 + 1, '.');
    rep (i, 0, sz(s)) t[i * 2 + 1] = s[i];
    int n = sz(t), m = 0, r = 0;
    vi p(n);
    rep (i, 0, n) {
        p[i] = (r > i ? min(r - i, p[m - (i - m)]) : 1);
        while (0 <= i - p[i] and i + p[i] < n and t[i - p[i]] == t[i + p[i]]) ++p[i];
        if (i + p[i] > r) m = i, r = i + p[i];
    }
    return p;
}
```

AhoCorasick.h

Description: Aho-Corasick automaton. To find all words, build a linked tree of patterns. To find the longest words that start at each position, reverse all input. Be careful of the extra node.
Time: $\mathcal{O}(\sigma N)$ construction, where $N = \text{sum of length of patterns}$

67b58d, 33 lines

```
struct AhoCorasick {
```

```
enum { sigma = 26, first = 'a' }; // change this!
struct Node {
    int fl, next[sigma];
    Node(int v) { memset(next, v, sizeof(next)); }
};
vector<Node> nd;
vi ord;
AhoCorasick() : nd(1, -1) {}
int insert(const string& s) {
    assert(!s.empty());
    int u = 0;
    for (char c : s) {
        int &v = nd[u].next[c - first];
        if (v == -1) { u = v = sz(nd); nd.emplace_back(-1); }
        else u = v;
    }
    return u;
}
void build() {
    nd[0].fl = sz(nd);
    nd.emplace_back(0);
    ord.push_back(0);
    rep (i, 0, sz(ord)) {
        int u = ord[i], pre = nd[u].fl;
        rep (c, 0, sigma) {
            int &v = nd[u].next[c], y = nd[pre].next[c];
            if (v == -1) v = y;
            else nd[v].fl = y, ord.push_back(v);
        }
    }
}
};
```

SuffixArray.h

Description: Builds suffix array for a string. $sa[i]$ is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n + 1$, and $sa[0] = n$. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: $lcp[i] = lcp(sa[i], sa[i-1])$, $lcp[0] = 0$. The input string must not contain any zero bytes.
Time: $\mathcal{O}(N \log N)$

a2a17d, 23 lines

```
struct SuffixArray {
    vi sa, lcp, rank;
    SuffixArray(string& s, int lim=256) {
        int n = sz(s) + 1, k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        x.push_back(0), sa = lcp = rank = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        rep(i,1,n) rank[sa[i]] = i;
        for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
            for (k && k--, j = sa[rank[i] - 1];
                 s[i + k] == s[j + k]; k++);
    }
};
```

SuffixAutomaton.h

Usage: Number of different substrings (sum $len[i] - len[link[i]]$)
Number of occurrences of string P (sum $cnt[i]$)
Shortest non-appearing string ($d[u] = 1 + \min d[v]$).
LCS (substring) of multiple strings.
Time: $\mathcal{O}(\sum |S|)$

1678f3, 55 lines

```
struct SuffixAutomaton {
    enum { sigma = 26, first = 'a' };
    struct Node {
        int len, link, next[sigma]; ll cnt;
        Node(int v) : len(0), link(0), cnt(0)
            { memset(next, v, sizeof(next)); }
    };
    vector<Node> nd;
    vi ord;
    SuffixAutomaton() : nd(1, -1) { nd[0].link = -1; }
    int insert(const string &s) {
        assert(!s.empty());
        int u = 0;
        for (char c : s) {
            int &v = nd[u].next[c - first];
            if (v == -1) { u = v = sz(nd); nd.emplace_back(-1); }
            else u = v;
            nd[u].cnt += 1;
        }
        return u;
    }
    int extend(int u, int c) {
        int v = nd[u].next[c];
        nd[v].len = nd[u].len + 1;
        int p = nd[u].link;
        while (p != -1 && nd[p].next[c] == -1)
            nd[p].next[c] = v, p = nd[p].link;
        if (p == -1) return nd[v].link = 0, v;
        int y = nd[p].next[c];
        if (nd[p].len + 1 == nd[y].len) return nd[v].link = y, v;
        int clone = sz(nd); nd.emplace_back(-1);
        rep (i, 0, sigma) nd[clone].next[i] =
            (nd[nd[y].next[i]].len > 0 ? nd[y].next[i] : 0);
        nd[clone].len = nd[p].len + 1;
        while (p != -1 && nd[p].next[c] == y)
            nd[p].next[c] = clone, p = nd[p].link;
        nd[nd[v].link = clone].link = nd[y].link;
        nd[y].link = clone;
        return v;
    }
    void build() {
        queue<int> q;
        for (q.push(0); !q.empty(); q.pop()) {
            int u = q.front();
            rep (c, 0, sigma) if (nd[u].next[c] != -1)
                q.push(extend(u, c));
        }
        vi cnt(sz(nd)); ord.resize(sz(nd) - 1);
        rep (i, 1, sz(nd)) ++cnt[nd[i].len];
        partial_sum(all(cnt), begin(cnt));
        rep (i, 1, sz(nd)) ord[--cnt[nd[i].len]] = i;
        for (int i = sz(ord) - 1; i >= 0; --i)
            nd[nd[ord[i]].link].cnt += nd[ord[i]].cnt;
    }
};
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: $\text{rotate}(v.\text{begin}(), v.\text{begin}() + \text{minRotation}(v), v.\text{end}());$
Time: $\mathcal{O}(N)$

d07a42, 8 lines

```
int minRotation(string s) {
```

```
int a=0, N=sz(s); s += s;
rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
}
return a;
}
```

LyndonFactorization.h

Description: Partition $S = w[0] + w[1] + \dots$ s.t. $w[0] \geq w[1] \geq \dots$ and $w[i]$ is < than all its cyclic shifts
Time: $\mathcal{O}(|S|)$

```
vector<pii> duval(const string& s) {
    vector<pii> res;
    for (int n = sz(s), i = 0, j, k; i < n; ) {
        for (j = i + 1, k = i; j < n && s[k] <= s[j]; ++j)
            k = (s[k] < s[j] ? i : k + 1);
        for (; i <= k; i += j - k) res.emplace_back(i, j - k);
    }
}
```

Hashing.h

Description: Airthmetic mod $2^{64} - 1$.

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on ull test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
    ull x; H(ull x=0) : x(x) {}
    H operator+(H o) { return x + o.x + (x + o.x < x); }
    H operator-(H o) { return *this + ~o.x; }
    H operator*(H o) { auto m = (uint128_t)x * o.x;
        return H((ull)m) + (ull)(m >> 64); }
    ull get() const { return x + !~x; }
    bool operator==(H o) const { return get() == o.get(); }
    bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (1ll)1e11+3; // (order ~ 3e9; random also ok)
```

```
struct HashInterval {
    vector<H> ha, pw;
    HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
        pw[0] = 1;
        rep(i,0,sz(str))
            ha[i+1] = ha[i] * C + str[i],
            pw[i+1] = pw[i] * C;
    }
    H hashInterval(int a, int b) { // hash [a, b)
        return ha[b] - ha[a] * pw[b - a];
    }
};
```

```
vector<H> getHashes(string& str, int length) {
    if (sz(str) < length) return {};
    H h = 0, pw = 1;
    rep(i,0,length)
        h = h * C + str[i], pw = pw * C;
    vector<H> ret = {h};
    rep(i,length,sz(str)) {
        ret.push_back(h = h * C + str[i] - pw * str[i-length]);
    }
    return ret;
}
```

```
H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

Graph (8)

8.1 Network flow

Dinic.h

Description: Flow algorithm with complexity $\mathcal{O}(VE \log U)$ where $U = \max|\text{cap}|$. $\mathcal{O}(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $\mathcal{O}(\sqrt{VE})$ for bipartite matching.

```
struct Dinic {
    struct E {
        int v, r;
        ll c, oc;
        // ll flow() { return max(oc - c, 0ll); }
    };
    int n;
    vi le, it, q;
    vector<vector<E>> adj;
    Dinic(int n): n(n), le(n), it(n), q(n), adj(n) {}
    void add(int u, int v, ll c, ll rc = 0) {
        adj[u].push_back({v, sz(adj[v]), c, c});
        adj[v].push_back({u, sz(adj[u]) - 1, rc, rc});
    }
    ll dfs(int u, int t, ll f) {
        if (u == t || !f) return f;
        for (int &i = it[u]; i < sz(adj[u]); ++i) {
            auto &[v, r, c, oc] = adj[u][i];
            if (le[v] == le[u] + 1)
                if (ll p = dfs(v, t, min(f, c)))
                    return c -= p, adj[v][r].c += p, p;
        }
        return 0;
    }
    ll flow(int s, int t) {
        ll res = 0; q[0] = s;
        rep(L,0,31) do { // rep(L,30,31) maybe faster for random
            le = it = vi(sz(q));
            int qi = 0, qe = le[s] = 1;
            while (qi < qe && !le[t]) {
                int u = q[qi++];
                for (auto [v, r, c, oc]: adj[u])
                    if (!le[v] && c >> (30 - L))
                        q[qi++] = v, le[v] = le[u] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) res += p;
        } while (le[t]);
        return res;
    }
    // bool inSCut(int u) { return le[u] != 0; }
};
```

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(V^2\sqrt{E})$

```
struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge*> cur;
    vector<vi> hs; vi H;
    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

    void addEdge(int s, int t, ll cap, ll rcap=0) {
```

```
    if (s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
    g[t].push_back({s, sz(g[s])-1, 0, rcap});
}

void addFlow(Edge& e, ll f) {
    Edge &back = g[e.dest][e.back];
    if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;
    back.f -= f; back.c += f; ec[back.dest] -= f;
}

ll calc(int s, int t) {
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;
    rep(i,0,v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);

    for (int hi = 0;;) {
        while (hs[hi].empty()) if (!hi--) return -ec[s];
        int u = hs[hi].back(); hs[hi].pop_back();
        while (ec[u] > 0) // discharge u
            if (cur[u] == g[u].data() + sz(g[u])) {
                H[u] = le9;
                for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
                    H[u] = H[e.dest]+1, cur[u] = &e;
                if (++co[H[u]], !--co[hi] && hi < v)
                    rep(i,0,v) if (hi < H[i] && H[i] < v)
                        --co[H[i]], H[i] = v + 1;
                hi = H[u];
            } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
                addFlow(*cur[u], min(ec[u], cur[u]->c));
            else ++cur[u];
    }
}

bool leftOfMinCut(int a) { return H[a] >= sz(g); }
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only. Sometimes using SPFA may be faster?

Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

```
#include <bits/extc++.h>

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
        ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
    }

    void path(int s) {
        fill(all(seen), 0);
```

```
fill(all(dist), INF);
dist[s] = 0; ll di;

__gnu_pbds::priority_queue<pair<ll, int>> q;
vector<decltype(q)::point_iterator> its(N);
q.push({ 0, s });

while (!q.empty()) {
    s = q.top().second; q.pop();
    seen[s] = 1; di = dist[s] + pi[s];
    for (edge& e : ed[s]) if (!seen[e.to]) {
        ll val = di - pi[e.to] + e.cost;
        if (e.cap - e.flow > 0 && val < dist[e.to]) {
            dist[e.to] = val;
            par[e.to] = &e;
            if (its[e.to] == q.end())
                its[e.to] = q.push({ -dist[e.to], e.to });
            else
                q.modify(its[e.to], { -dist[e.to], e.to });
        }
    }
    rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        ll fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
        rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
        return {totflow, totcost/2};
    }
}
```

```
// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i,0,N) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}

};
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time: $\mathcal{O}(V^3)$

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
```

```
vector<vi> co(n);
rep(i,0,n) co[i] = {i};
rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { //  $\mathcal{O}(V^2) \rightarrow \mathcal{O}(E \log V)$  with prio. queue
        w[t] = INT_MIN;
        s = t, t = max_element(all(w)) - w.begin();
        rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
}
return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h"0418b3, 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
    vector<Edge> tree;
    vi par(N);
    rep(i,1,N) {
        PushRelabel D(N); // Dinic also works
        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        rep(j,i+1,N)
            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
    }
    return tree;
}
```

8.2 Matching

hopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: $vi\ btoa(m, -1);$ hopcroftKarp($g, btoa$);
Time: $\mathcal{O}(\sqrt{E})$

f612e4, 42 lines

```
bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi& B) {
    if (A[a] != L) return 0;
    A[a] = -1;
    for (int b : g[a]) if (B[b] == L + 1) {
        B[b] = 0;
        if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B))
            return btoa[b] = a, 1;
    }
    return 0;
}
```

```
int hopcroftKarp(vector<vi>& g, vi& btoa) {
    int res = 0;
    vi A(g.size()), B(btoa.size()), cur, next;
    for (;;) {
        fill(all(A), 0);
        fill(all(B), 0);
        cur.clear();
        for (int a : btoa) if (a != -1) A[a] = -1;
        rep(a,0,sz(g)) if (A[a] == 0) cur.push_back(a);
```

```
for (int lay = 1;; lay++) {
    bool islast = 0;
    next.clear();
    for (int a : cur) for (int b : g[a]) {
        if (btoa[b] == -1) {
            B[b] = lay;
            islast = 1;
        }
        else if (btoa[b] != a && !B[b]) {
            B[b] = lay;
            next.push_back(btoa[b]);
        }
    }
    if (islast) break;
    if (next.empty()) return res;
    for (int a : next) A[a] = lay;
    cur.swap(next);
}
rep(a,0,sz(g))
    res += dfs(a, 0, g, btoa, A, B);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"DFSMatching.h"da4196, 20 lines

```
vi cover(vector<vi>& g, int n, int m) {
    vi match(m, -1);
    int res = dfsMatching(g, match);
    vector<bool> lfound(n, true), seen(m);
    for (int it : match) if (it != -1) lfound[it] = false;
    vi q, cover;
    rep(i,0,n) if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back(); q.pop_back();
        lfound[i] = 1;
        for (int e : g[i]) if (!seen[e] && match[e] != -1) {
            seen[e] = true;
            q.push_back(match[e]);
        }
    }
    rep(i,0,n) if (!lfound[i]) cover.push_back(i);
    rep(i,0,m) if (seen[i]) cover.push_back(n+i);
    assert(sz(cover) == res);
    return cover;
}
```

WeightedMatching.h

Description: Given weight matrix, find maximum perfect matching.
Time: $\mathcal{O}(N^3)$

204606, 35 lines

```
const ll INF = numeric_limits<ll>::max() / 4;
struct KM {
    int n, l, r; ll ans; // fl and fr are the match
    vector<ll> hl, hr; vector<int> fl, fr, pre, q;
    void bfs(const vector<ll>& &w, int s) {
        vector<int> vl(n), vr(n); vector<ll> slk(n, INF);
        l = r = 0; vr[q[r++]] = s = true;
        auto check = [&](int x) -> bool {
            if (vl[x] || slk[x] > 0) return true;
            vl[x] = true; slk[x] = INF;
            if (fl[x] != -1) return (vr[q[r++]] = fl[x]) = true;
            while (x != -1) swap(x, fr[fl[x] = pre[x]]);
            return false;
        };
        while (true) {
            while (l < r)
```

```
for (int x = 0, y = q[l++]; x < n; ++x) if (!vl[x])
    if (slk[x] > hl[x] + hr[y] - w[x][y]) {
        slk[x] = hl[x] + hr[y] - w[x][y];
        if (pre[x] = y, !check(x)) return;
    }
ll d = *min_element(all(slk));
for (int x = 0; x < n; ++x)
    vl[x] ? hl[x] += d : slk[x] -= d;
for (int x = 0; x < n; ++x) if (vr[x]) hr[x] -= d;
for (int x = 0; x < n; ++x) if (!check(x)) return;
}
}
KM(int n_, const vector<vector<ll>> &w) : n(n_), ans(0),
    hl(n), hr(n), fl(n, -1), fr(fl), pre(n), q(n) {
    for (int i = 0; i < n; ++i) hl[i]=*max_element(all(w[i]));
    for (int i = 0; i < n; ++i) bfs(w, i);
    for (int i = 0; i < n; ++i) ans += w[i][fl[i]];
}
};
```

GeneralMatching.h

Description: Matching for general graphs.
Time: $\mathcal{O}(N^3)$

58ebf5, 52 lines

```
struct Matching {
    queue<int> q; int n;
    vector<int> fa, s, vis, pre, match;
    vector<vector<int>> G;
    Matching(int _n) : n(_n), fa(n + 1), s(n + 1), vis(n + 1),
        pre(n + 1, n), match(n + 1, n), G(n) {}
    void add_edge(int u, int v)
    { G[u].push_back(v), G[v].push_back(u); }
    int Find(int u)
    { return u == fa[u] ? u : fa[u] = Find(fa[u]); }
    int LCA(int x, int y) {
        static int tk = 0; tk++; x = Find(x); y = Find(y);
        for (; swap(x, y)) if (x != n) {
            if (vis[x] == tk) return x;
            vis[x] = tk;
            x = Find(pre[match[x]]);
        }
    }
    void Blossom(int x, int y, int l) {
        for (; Find(x) != l; x = pre[y]) {
            pre[x] = y, y = match[x];
            if (s[y] == l) q.push(y), s[y] = 0;
            for (int z: {x, y}) if (fa[z] == z) fa[z] = l;
        }
    }
    bool Bfs(int r) {
        iota(all(fa), 0); fill(all(s), -1);
        q = queue<int>(); q.push(r); s[r] = 0;
        for (; !q.empty(); q.pop()) {
            for (int x = q.front(); int u : G[x])
                if (s[u] == -1) {
                    if (pre[u] = x, s[u] = 1, match[u] == n) {
                        for (int a = u, b = x, last;
                            b != n; a = last, b = pre[a])
                            last = match[b], match[b] = a, match[a] = b;
                        return true;
                    }
                    q.push(match[u]); s[match[u]] = 0;
                } else if (!s[u] && Find(u) != Find(x)) {
                    int l = LCA(u, x);
                    Blossom(x, u, l); Blossom(u, x, l);
                }
            }
        }
        return false;
    }
};
```

```
int solve() {
    int ans = 0;
    for (int x = 0; x < n; ++x)
        if (match[x] == n) ans += Bfs(x);
    return ans;
}
};
```

8.3 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.

Time: $\mathcal{O}(E + V)$

76b5c9, 24 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
    int low = val[j] = ++Time, x; z.push_back(j);
    for (auto e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ?: dfs(e, g, f));

    if (low == val[j]) {
        do {
            x = z.back(); z.pop_back();
            comp[x] = ncomps;
            cont.push_back(x);
        } while (x != j);
        f(cont); cont.clear();
        ncomps++;
    }
    return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
    int n = sz(g);
    val.assign(n, 0); comp.assign(n, -1);
    Time = ncomps = 0;
    rep(i, 0, n) if (comp[i] < 0) dfs(i, g, f);
}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two distinct paths between any two nodes. Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage: int eid = 0; ed.resize(N); for each edge (a,b) { ed[a].emplace_back(b, eid); ed[b].emplace_back(a, eid++); } bicomps([&](const vi& edgelist) {...});

Time: $\mathcal{O}(E + V)$

c6b7c7, 32 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
    int me = num[at] = ++Time, top = me;
    for (auto [y, e] : ed[at]) if (e != par) {
        if (num[y]) {
            top = min(top, num[y]);
            if (num[y] < me)
                st.push_back(e);
        } else {
```

```
int si = sz(st);
int up = dfs(y, e, f);
top = min(top, up);
if (up == me) {
    st.push_back(e);
    f(vi(st.begin() + si, st.end()));
    st.resize(si);
}
else if (up < me) st.push_back(e);
else { /* e is a bridge */ }
}
}
return top;
}

template<class F>
void bicomps(F f) {
    num.assign(sz(ed), 0);
    rep(i, 0, sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage: TwoSat ts(number of boolean variables); ts.either(0, ~3); // Var 0 is true or var 3 is false ts.setValue(2); // Var 2 is true ts.atMostOne({0, ~1, 2}); // <= 1 of vars 0, ~1 and 2 are true ts.solve(); // Returns true iff it is solvable ts.values[0..N-1] holds the assigned values to the vars

Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 56 lines

```
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }

    void either(int f, int j) {
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }
    void setValue(int x) { either(x, x); }

    void atMostOne(const vi& li) { // (optional)
        if (sz(li) <= 1) return;
        int cur = ~li[0];
        rep(i, 2, sz(li)) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }
}
```

```
vi val, comp, z; int time = 0;
int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
        low = min(low, val[e] ? : dfs(e));
    if (low == val[i]) do {
        x = z.back(); z.pop_back();
        comp[x] = low;
        if (values[x]>>1] == -1)
            values[x]>>1] = x&1;
    } while (x != i);
    return val[i] = low;
}

bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
}
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Time: $\mathcal{O}(V + E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
    int n = sz(gr);
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        if (it == end){ ret.push_back(x); s.pop_back(); continue; }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1; s.push_back(y);
        }
    }
    for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
    return {ret.rbegin(), ret.rend()};
}
```

DominatorTree.h

Description: Build dominator tree of graph adj. 0 base. Root’s dominator is itself, and unreachable nodes have -1 as dominator.

Time: $\mathcal{O}(M \log N)$

ac6fad, 41 lines

```
struct DominatorTree {
    vi p, semi, ord, dom, f, val;
    vector<vi> adj, pre, bkt;
    void dfs(int u) {
        semi[u] = sz(ord);
        ord.push_back(u);
        for (int v : adj[u]) {
            if (semi[v] == -1) p[v] = u, dfs(v);
            pre[v].push_back(u);
        }
    }
    int eval(int u, int t = 0) {
        if (f[u] == -1) return t ? -1 : u;
        if (int p = eval(f[u], 1); p != -1) {
            if (semi[val[f[u]]] < semi[val[u]])
                val[u] = val[f[u]];
            f[u] = p;
        }
    }
};
```

```
        return t ? p : val[u];
    } return t ? f[u] : val[u];
}

DominatorTree(int N, const vector<vi>& adj, int r): p(N, -1),
semi(p), dom(p), f(p), val(N), adj(adj), pre(N), bkt(N) {
    iota(all(val), 0);
    dfs(r);
    for (int i = sz(ord); --i; ) {
        int u = ord[i];
        for (int v : pre[u])
            semi[u] = min(semi[u], semi[eval(v)]);
        bkt[ord[semi[u]]].push_back(u);
        f[u] = p[u];
        for (int v : bkt[p[u]]) {
            int w = eval(v);
            dom[v] = semi[w] < semi[v] ? w : p[u];
        }
        bkt[p[u]].clear();
    }
    dom[r] = r;
    for (int u: ord) if (dom[u] != ord[semi[u]])
        dom[u] = dom[dom[u]];
}
};
```

3.4 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : {fan[0], u, end})
            for (int& z = free[y] = 0; adj[y][z] != -1; z++);
    }
    rep(i,0,sz(eds))
        for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
    return ret;
}
```

8.5 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
    if (!P.any()) { if (!X.any()) f(R); return; }
    auto q = (P | X)._Find_first();
    auto cands = P & ~eds[q];
    rep(i,0,sz(eds)) if (cands[i]) {
        R[i] = 1;
        cliques(eds, f, P & eds[i], X & eds[i], R);
        R[i] = P[i] = 0; X[i] = 1;
    }
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit=0.025, pk=0;
    struct Vertex { int i, d=0; };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv& r) {
        for (auto& v : r) v.d = 0;
        for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv& R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
        while (sz(R)) {
            if (sz(q) + R.back().d <= sz(qmax)) return;
            q.push_back(R.back().i);
            vv T;
            for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
            if (sz(T)) {
                if (S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
                C[1].clear(), C[2].clear();
                for (auto v : T) {
                    int k = 1;
                    auto f = [&](int i) { return e[v.i][i]; };
                    while (any_of(all(C[k]), f)) k++;
                    if (k > mxk) mxk = k, C[mxk + 1].clear();
                    if (k < mnk) T[j++].i = v.i;
                    C[k].push_back(v.i);
                }
                if (j > 0) T[j - 1].d = 0;
                rep(k,mnk,mxk + 1) for (int i : C[k])
                    T[j].i = i, T[j++].d = k;
                expand(T, lev + 1);
            } else if (sz(q) > sz(qmax)) qmax = q;
            q.pop_back(), R.pop_back();
        }
    }
};
```

```
    }
}
vi maxClique() { init(V), expand(V); return qmax; }
Maxclique(vb connn) : e(connn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
}
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

8.6 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.
Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

```
vector<vi> treeJump(vi& P){
    int on = 1, d = 1;
    while(on < sz(P)) on *= 2, d++;
    vector<vi> jmp(d, P);
    rep(i,1,d) rep(j,0,sz(P))
        jmp[i][j] = jmp[i-1][jmp[i-1][j]];
    return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
    rep(i,0,sz(tbl))
        if(steps&(1<<i)) nod = tbl[i][nod];
    return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = sz(tbl); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d) a = c, b = d;
    }
    return tbl[0][a];
}
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(|S| \log |S|)$

```
"LCA.h"
9775a0, 21 lines

typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
    static vi rev; rev.resize(sz(lca.time));
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li)-1;
    rep(i,0,m) {
        int a = li[i], b = li[i+1];
        li.push_back(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    rep(i,0,sz(li)) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
```

```
    rep(i,0,sz(li)-1) {
        int a = li[i], b = li[i+1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }
    return ret;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
Time: $\mathcal{O}((\log N)^2)$

```
"../data-structures/LazySegmentTree.h"
cbb1fc, 46 lines

template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, depth, rt, pos;
    SGT<Val, Tag> tree;
    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1), depth(N),
          rt(N),pos(N),tree(N){ dfsSz(0); dfsHld(0); }
    void dfsSz(int v) {
        if (par[v] != -1) adj[v].erase(find(all(adj[v]), par[v]));
        for (int& u : adj[v]) {
            par[u] = v, depth[u] = depth[v] + 1;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (; rt[u] != rt[v]; v = par[rt[v]]) {
            if (depth[rt[u]] > depth[rt[v]]) swap(u, v);
            op(pos[rt[v]], pos[v] + 1);
        }
        if (depth[u] > depth[v]) swap(u, v);
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, int val) {
        process(u, v, [&](int l, int r) { tree.modify(l, r, val); });
    }
    int queryPath(int u, int v) { // Modify depending on problem
        int res = -le9;
        process(u, v, [&](int l, int r) {
            res = max(res, tree.query(l, r).v);
        });
        return res;
    }
    int querySubtree(int v) { // modifySubtree is similar
        return tree.query(pos[v] + VALS_EDGES, pos[v] + siz[v]).v;
    }
};
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.
Time: All operations take amortized $\mathcal{O}(\log N)$.

```
c54b60, 101 lines

struct Node { // Splay tree. Root's pp contains tree's parent.
    Node *p = 0, *pp = 0, *c[2];
    bool flip = 0;
    Node() { c[0] = c[1] = 0; fix(); }
    void fix() {
        if (c[0]) c[0]->p = this;
        if (c[1]) c[1]->p = this;
        // (+ update sum of subtree elements etc. if wanted)
    }
    void pushFlip() {
        if (!flip) return;
        flip = 0; swap(c[0], c[1]);
        if (c[0]) c[0]->flip ^= 1;
        if (c[1]) c[1]->flip ^= 1;
    }
    int up() { return p ? p->c[1] == this : -1; }
    void rot(int i, int b) {
        int h = i ^ b;
        Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
        if ((y->p = p)) p->c[up()] = y;
        c[i] = z->c[i ^ 1];
        if (b < 2) {
            x->c[h] = y->c[h ^ 1];
            y->c[h ^ 1] = x;
        }
        z->c[i ^ 1] = this;
        fix(); x->fix(); y->fix();
        if (p) p->fix();
        swap(pp, y->pp);
    }
    void splay() {
        for (pushFlip(); p; ) {
            if (p->p) p->p->pushFlip();
            p->pushFlip(); pushFlip();
            int c1 = up(), c2 = p->up();
            if (c2 == -1) p->rot(c1, 2);
            else p->p->rot(c2, c1 != c2);
        }
    }
    Node* first() {
        pushFlip();
        return c[0] ? c[0]->first() : (splay(), this);
    }
};

struct LinkCut {
    vector<Node> node;
    LinkCut(int N) : node(N) {}

    void link(int u, int v) { // add an edge (u, v)
        assert(!connected(u, v));
        makeRoot(&node[u]);
        node[u].pp = &node[v];
    }
    void cut(int u, int v) { // remove an edge (u, v)
        Node *x = &node[u], *top = &node[v];
        makeRoot(top); x->splay();
        assert(top == (x->pp ? x->c[0]));
        if (x->pp) x->pp = 0;
        else {
            x->c[0] = top->p = 0;
            x->fix();
        }
    }
};
```

```

}
bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
}
void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
        u->c[0]->p = 0;
        u->c[0]->flip ^= 1;
        u->c[0]->pp = u;
        u->c[0] = 0;
        u->fix();
    }
}
void push_down(Node* u){
    u->pushFlip();
    if(u->c[0])push_down(u->c[0]);
    if(u->c[1])push_down(u->c[1]);
}
Node* access(Node* u) {
    u->splay();
    if(u->c[1]){
        u->c[1]->p=0;
        u->c[1]->pp=u;
        u->c[1]=0;
        u->fix();
    }
    while (Node* pp = u->pp) {
        pp->splay(); u->pp = 0;
        if (pp->c[1]) {
            pp->c[1]->p = 0; pp->c[1]->pp = pp; }
        pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
}
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E\log V)$

../data-structures/UnionFindRollback.h"	39e620, 60 lines
<pre>struct Edge { int a, b; ll w; }; struct Node { Edge key; Node *l, *r; ll delta; void prop() { key.w += delta; if (l) l->delta += delta; if (r) r->delta += delta; delta = 0; } Edge top() { prop(); return key; } }; Node *merge(Node *a, Node *b) { if (!a !b) return a ?: b; a->prop(), b->prop(); if (a->key.w > b->key.w) swap(a, b); swap(a->l, (a->r = merge(b, a->r))); return a; } void pop(Node& a) { a->prop(); a = merge(a->l, a->r); }</pre>	
<pre>pair<ll, vi> dmst(int n, int r, vector<Edge>& g) { RollbackUF uf(n); vector<Node*> heap(n);</pre>	

```

for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
ll res = 0;
vi seen(n, -1), path(n), par(n);
seen[r] = r;
vector<Edge> Q(n), in(n, {-1,-1}), comp;
deque<tuple<int, int, vector<Edge>>> cyps;
rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
        if (!heap[u]) return {-1,{};};
        Edge e = heap[u]->top();
        heap[u]->delta -= e.w, pop(heap[u]);
        Q[qi] = e, path[qi++] = u, seen[u] = s;
        res += e.w, u = uf.find(e.a);
        if (seen[u] == s) {
            Node* cyc = 0;
            int end = qi, time = uf.time();
            do cyc = merge(cyc, heap[w = path[--qi]]);
            while (uf.join(u, w));
            u = uf.find(u), heap[u] = cyc, seen[u] = -1;
            cyps.push_front({u, time, {Q[qi], &Q[end]}});
        }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
}

for (auto& [u,t,comp] : cyps) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
}
rep(i,0,n) par[i] = in[i].a;
return {res, par};
}
```

8.7 Math

8.7.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $mat[a][b]--$, $mat[b][b]++$ (and $mat[b][a]--$, $mat[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

8.7.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Data structures (9)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n 'th element, and finding the index of an element. To get a map, change `nulltype`.
Time: $\mathcal{O}(\log N)$

#include <bits/extc++.h> using namespace __gnu_pbds;	782797, 16 lines
---	------------------

```

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

void example() {
    Tree<int> t, t2; t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: Hash map with mostly the same API as `unordered_map`, but $\sim 3\times$ faster. Uses $1.5\times$ memory. Initial capacity must be a power of 2 (if provided).

#include <bits/extc++.h> // To use most bits rather than just the lowest ones: struct chash { // large odd number for C const uint64_t C = 11(4e18 * acos(0)) 71; ll operator()(ll x) const { return __builtin_bswap64(x*C); } }; __gnu_pbds::gp_hash_table<ll,int,chash> h({}, {}, {}, {}, {1<16});	d77092, 7 lines
--	-----------------

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip (A)'s: `st`, `time()` and `rollback()`.
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

struct RollbackUF { vi e; vector<pii> st; // (A) RollbackUF(int n) : e(n, -1) {} int size(int x) { return -e[find(x)]; } int find(int x) { return e[x] < 0 ? x : find(e[x]); } int time() { return sz(st); } // (A) void rollback(int t) { // (A) for (int i = time(); i --> t;) e[st[i].first] = st[i].second; st.resize(t); } bool join(int a, int b) { a = find(a), b = find(b); if (a == b) return false; if (e[a] > e[b]) swap(a, b); st.push_back({a, e[a]}); // (A) st.push_back({b, e[b]}); // (A) e[a] += e[b]; e[b] = a; return true; } };	de4ad0, 21 lines
--	------------------

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).
Time: $\mathcal{O}(\log N)$

struct Line { mutable ll k, m, p; // minimum: change to k > o.k; bool operator<(const Line& o) const { return k < o.k; } bool operator<(ll x) const { return p < x; } }; struct LineContainer : multiset<Line, less<>> { // (for doubles, use inf = 1/.0, div(a,b) = a/b)	8ec1c7, 30 lines
---	------------------

```

static const ll inf = LLONG_MAX;
ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf; // <
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
}
void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
        isect(x, erase(y));
}
ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
}
};

```

LazySegmentTree.h

Description: ZKW with ACL style nodes. [l, r), 0-base.

Usage: SGT<Val, Tag> sgt(n);

Time: $\mathcal{O}(N + Q \log N)$.

0f94ce, 55 lines

```

struct Val {
    int v;
    Val(int v = 0) : v(v) {} // must return identity element
    Val operator + (const Val& o) const {
        return Val(max(v, o.v)); }
    // merge two Vals, order is important
};
struct Tag {
    int t;
    Tag(int t = 0) : t(t) {} // must return identity element
    Tag operator + (const Tag& o) const { return Tag(t + o.t); }
    // compose two Tags, order is important
    Val operator() (Val v) const { return Val(v.v + t); }
    // apply the Tag to v
};

int bc(int u) { return u <= 1 ? 1 : (2 << __lg(u-1)); }
template <class V, class T> struct SGT {
    int n; vector<V> val; vector<T> tag;
    SGT(int _n): n(bc(_n)), val(n*2), tag(n*2) {}
    SGT(const vector<V>& v): n(bc(sz(v))), val(n*2), tag(n*2) {
        rep (i, 0, sz(v)) val[i+n] = v[i];
        for (int i = n; --i; ) val[i] = val[i*2] + val[i*2+1];
    }
    void upd(int u, T t)
    { val[u] = t(val[u]); if (u < n) tag[u] = tag[u] + t; }
    void pull(int u)
    { while (u /= 2) val[u] = tag[u](val[u*2] + val[u*2+1]); }
    void push(int u) {
        for (int h = __lg(n)+1, i; --h;) {
            i = u >> h;
            upd(i * 2, tag[i]);
            upd(i * 2 + 1, tag[i]);
            tag[i] = T();
        }
    }
    void set(int p, V v)
    { push(p += n); val[p] = v; pull(p); }
    V query(int l, int r) {
        V rl, rr;
        for (push(l+=n), push((r+=n)-1); l < r; l /= 2, r /= 2) {

```

```

            if (l & 1) rl = rl + val[l++];
            if (r & 1) rr = val[--r] + rr;
        }
        return rl + rr;
    }
    void modify(int l, int r, T t) {
        int tl = (l += n), tr = (r += n) - 1;
        for (push(tl), push(tr); l < r; l >= 1, r >= 1) {
            if (l & 1) upd(l++, t);
            if (r & 1) upd(--r, t);
        }
        pull(tl); pull(tr);
    }
};

```

LiChao.h

Description: Add segment

Time: $\mathcal{O}(N + Q \log N)$.

982e97, 59 lines

```

constexpr ll LINF = std::numeric_limits<ll>::max() / 2;
struct line {
    ll a, b;
    ll operator() (const ll x) const {
        return a * x + b;
    }
};
struct lichao {
    size_t size;
    vector<ll> xs;
    vector<size_t> left, right;
    vector<line> best;
    explicit lichao(const vector<ll>& xs_) : xs(xs_) {
        sort(begin(xs), end(xs));
        xs.erase(unique(begin(xs), end(xs)), end(xs));
        const size_t n = xs.size();
        size = 1;
        while (size < n) size <= 1;
        xs.resize(size, xs.back());
        left.resize(2 * size);
        right.resize(2 * size);
        best.resize(2 * size, {0, LINF});
        for (size_t i = 0; i < size; ++i)
            left[i + size] = right[i + size] = i;
        for (size_t i = size - 1; i > 0; --i) {
            left[i] = left[i << 1];
            right[i] = right[i << 1 | 1];
        }
    }
    inline size_t get_id(ll p) const {
        return lower_bound(all(xs), p) - begin(xs) + size;
    }
    void add_segment(const ll l, const ll r, const line& add) {
        size_t a = get_id(l), b = get_id(r);
        while (a < b) {
            if (a & 1) add_line(a++, add);
            if (b & 1) add_line(--b, add);
            a >= 1, b >= 1;
        }
    }
    ll que(const ll x) {
        size_t k = get_id(x);
        ll ret = LINF;
        while (k > 0) {
            ret = min(ret, best[k](x));
            k >>= 1;
        }
        return ret;
    }
    void add_line(size_t k, line add) {

```

```

        const size_t l = left[k];
        const size_t r = right[k];
        const size_t m = (l + r) / 2;
        auto& cur = best[k];
        if (add(xs[m]) < cur(xs[m])) swap(add, cur);
        if (add(xs[l]) < cur(xs[l])) add_line(k << 1, add);
        if (add(xs[r]) < cur(xs[r])) add_line(k << 1 | 1, add);
    }
};

```

Treap.h

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.

Time: $\mathcal{O}(\log N)$

dd38d5, 50 lines

```

struct Node {
    Node *l = 0, *r = 0;
    int val, c = 1, y; // maybe not use rand()!
    Node(int val) : val(val), y(rand()) {}
    void pull();
};
int cnt(Node* n) { return n ? n->c : 0; }
void Node::pull() { c = cnt(l) + cnt(r) + 1; }
template<class F> void each(Node* n, F f)
{ if (n) { each(n->l, f); f(n->val); each(n->r, f); } }

pair<Node*, Node*> split(Node* n, int k) {
    if (!n) return {};
    if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
        auto pa = split(n->l, k);
        n->l = pa.second;
        n->pull();
        return {pa.first, n};
    }
    auto pa = split(n->r, k - cnt(n->l) - 1); // and just "k"
    n->r = pa.first;
    n->pull();
    return {n, pa.second};
}

Node* merge(Node* l, Node* r) {
    if (!l) return r;
    if (!r) return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        l->pull();
        return l;
    }
    r->l = merge(l, r->l);
    r->pull();
    return r;
}

Node* ins(Node* t, Node* n, int pos) { // 0-base
    auto pa = split(t, pos);
    return merge(merge(pa.first, n), pa.second);
}

// Example application: move the range [l, r) to index k
// void move(Node& t, int l, int r, int k) {
//     Node *a, *b, *c;
//     tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
//     if (k <= l) t = merge(ins(a, b, k), c);
//     else t = merge(a, ins(c, b, k - r));
// }

```

RMQ.h
Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage: RMQ rmq(values); rmq.query(inclusive, exclusive);
Time: $\mathcal{O}(|V|\log|V| + Q)$

05c3f0, 16 lines

```
template <class T>
struct RMQ {
    vector<vector<T>>> a;
    RMQ(const vector<T>& v) : a(1, v) {
        for (int p = 1, k = 1; p * 2 <= sz(v); p *= 2, ++k) {
            a.emplace_back(sz(v) - p * 2 + 1);
            rep(j, 0, sz(a[k]))
                a[k][j] = min(a[k - 1][j], a[k - 1][j + p]);
        }
    }
    T query(int l, int r) {
        assert(1 < r);
        int d = 31 - __builtin_clz(r - 1);
        return min(a[d][l], a[d][r - (1 << d)]);
    }
};
```

MoQueries.h
Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).
Time: $\mathcal{O}(N\sqrt{Q})$

a12ef4, 49 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) {
        pii q = Q[qi];
        while (L > q.first) add(--L, 0);
        while (R < q.second) add(R++, 1);
        while (L < q.first) del(L++, 0);
        while (R > q.second) del(--R, 1);
        res[qi] = calc();
    }
    return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0) {
    int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
    add(0, 0), in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto& f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep) I[x] = N++;
        for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
        if (!dep) I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);
    #define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) rep(end, 0, 2) {
        int &a = pos[end], b = Q[qi][end], i = 0;
        #define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
```

```
        else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
        I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
    }
    return res;
}
```

Various (10)

10.1 Intervals

IntervalContainer.h
Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Time: $\mathcal{O}(\log N)$

edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}
```

```
void removeInterval(set<pii>& is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L) is.erase(it);
    else (int&)it->second = L;
    if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h
Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty).
Time: $\mathcal{O}(N \log N)$

9e9d8d, 19 lines

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while (at < sz(I) && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if (mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
}
```

ConstantIntervals.h
Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.
Usage: constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});
Time: $\mathcal{O}(k \log \frac{n}{k})$

753a4c, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
    if (p == q) return;
    if (from == to) {
        g(i, to, p);
        i = to; p = q;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid+1, to, f, g, i, p, q);
    }
}

template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
    if (to <= from) return;
    int i = from; auto p = f(i), q = f(to-1);
    rec(from, to-1, f, g, i, p, q);
    g(i, to, q);
}
```

10.2 Misc. algorithms

TernarySearch.h
Description: Find the smallest i in [a, b] that maximizes f(i), assuming that f(a) < ... < f(i) ≥ ... ≥ f(b). To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).
Usage: int ind = ternSearch(0, n-1, [&](int i){return a[i];});
Time: $\mathcal{O}(\log(b - a))$

9155b4, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
        if (f(mid) < f(mid+1)) a = mid; // (A)
        else b = mid+1;
    }
    rep(i, a+1, b+1) if (f(a) < f(i)) a = i; // (B)
    return a;
}
```

FastKnapsack.h
Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S ≤ t such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$

b20ccc, 16 lines

```
int knapsack(vi w, int t) {
    int a = 0, b = 0, x;
    while (b < sz(w) && a + w[b] <= t) a += w[b++];
    if (b == sz(w)) return a;
    int m = *max_element(all(w));
    vi u, v(2*m, -1);
    v[a+m-t] = b;
    rep(i, b, sz(w)) {
        u = v;
        rep(x, 0, m) v[x+w[i]] = max(v[x+w[i]], u[x]);
        for (x = 2*m; --x > m;) rep(j, max(0, u[x]), v[x])
```

```
        v[x-w[j]] = max(v[x-w[j]], j);
    }
    for (a = t; v[a+m-t] < 0; a--);
    return a;
}
```

SimulatedAnneal.h
Description: rnd() should return $r \in [0, 1]$. 63137d, 10 lines

```
void simulateAnneal() {
    double t = 100000, now = ans;
    while (t > 0.001) {
        double nxt = now + t * rnd();
        double delta = calc(nxt) - calc(now);
        if (exp(-delta / t) > rnd()) now = nxt;
        t *= 0.97;
    }
    rep (i, 0, 1000) calc(ans + t * rnd());
}
```

10.3 Dynamic programming

KnuthDP.h
Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j - 1]$ and $p[i + 1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.
Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h
Description: Given $a[i] = \min_{l \circ(i) \leq k < h i(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for $i = L..R - 1$.
Time: $\mathcal{O}((N + (hi - lo)) \log N)$ d38d2b, 18 lines

```
struct DP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best (LLONG_MAX, LO);
        rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second+1);
        rec(mid+1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

ConvexQuadAliens.h
Time: $\mathcal{O}(N \log N \log (rb - lb))$ af02bc, 47 lines

```
vector<pair<ll, int>> dp(N + 1); // (min cost, cnt)
deque<pii> dq; // (source, best until)
const auto quad_alien = [&](int n, int k) -> ll {
    const auto f = [&](int j, int i, ll pen) -> pair<ll, int> {
        return { dp[j].first + cost(j, i) + pen,
            dp[j].second + 1 }; };
    const auto find_t = [&](int u, int v, int lb, int rb) {
        rb += 1;
        while (lb < rb) {
            int mid = (lb + rb) / 2;
```

```
            if (f(u, mid, 0) <= f(v, mid, 0)) lb = mid + 1;
            else rb = mid;
        }
        return lb;
    };
    const auto solve = [&](const ll pen) -> void {
        dp[0] = pair<ll, int>(0, 0);
        dq.clear(); dq.emplace_back(0, n);
        rep (i, 1, n + 1) {
            while (dq.front().second < i) dq.pop_front();
            dp[i] = f(dq.front().first, i, pen);
            int mid = -1;
            while (!dq.empty()) {
                int lb = (sz(dq) == 1 ? dq.back().first : end(dq)[-2].second + 1);
                if (f(dq.back().first, lb, 0) >= f(i, lb, 0)) {
                    dq.pop_back(); continue; }
                mid = find_t(dq.back().first, i, lb, dq.back().second);
                if (mid <= lb) dq.pop_back(); else break;
            }
            if (!dq.empty() && mid <= n) {
                dq.back().second = mid - 1;
                dq.emplace_back(i, n);
            } else if (dq.empty())
                dq.emplace_back(i, n);
        }
    };
    // beware fp precision / range
    ll lb = 0, rb = 1e11;
    while (lb < rb) {
        ll mid = (lb + rb) / 2;
        solve(mid);
        if (dp[n].second > k) lb = mid + 1;
        else rb = mid;
    }
    solve(lb);
    return dp[n].first - lb * k;
};
```

10.4 Debugging tricks

- signal(SIGSEGV, [] (int) { _Exit(0); }); converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). _GLIBCXX_DEBUG failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- feenableexcept (29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks
__builtin_ia32_ldmxcsr(40896); disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- x & -x is the least bit in x.
- for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- c = x&-x, r = x+c; (((r^x) >> 2)/c) | r is the next number after x with the same number of bits set.
- rep(b, 0, K) rep(i, 0, (1 << K))

```
        if (i & 1 << b) D[i] += D[i^(1 << b)];
    computes all sums of subsets.
```

10.5.2 Pragmas

- #pragma GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- #pragma GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.
- #pragma GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

FastMod.h
Description: Compute $a \% b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0, 2b)$. 751a02, 8 lines

```
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

FastInput.h
Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: ./a.out < input.txt
Time: About 5x as fast as cin/scanf. 7b3c70, 17 lines

```
inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // returns 0 on EOF
}

int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-' ) return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 48;
    return a - 48;
}
```

Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients
Pigeon-hole principle
Inclusion/exclusion
Catalan number
Pick's theorem
Number theory
Integer parts
Divisibility
Euclidean algorithm
Modular arithmetic
* Modular multiplication
* Modular inverses
* Modular exponentiation by squaring
Chinese remainder theorem
Fermat's little theorem
Euler's theorem
Phi function
Frobenius number
Quadratic reciprocity
Pollard-Rho
Miller-Rabin
Hensel lifting
Vieta root jumping
Game theory
Combinatorial games
Game trees
Mini-max
Nim
Games on graphs
Games on graphs with loops
Grundy numbers
Bipartite games without repetition
General games without repetition
Alpha-beta pruning
Probability theory
Optimization
Binary search
Ternary search
Unimodality and convex functions
Binary search on derivative
Numerical methods
Numeric integration
Newton's method
Root-finding with binary/ternary search
Golden section search
Matrices
Gaussian elimination
Exponentiation by squaring
Sorting
Radix sort
Geometry
Coordinates and vectors
* Cross product
* Scalar product
Convex hull
Polygon cut
Closest pair
Coordinate-compression
Quadtrees
KD-trees
All segment-segment intersection
Sweeping
Discretization (convert to events and sweep)
Angle sweeping
Line sweeping
Discrete second derivatives
Strings
Longest common substring
Palindrome subsequences

Knuth-Morris-Pratt
Tries
Rolling polynomial hashes
Suffix array
Suffix tree
Aho-Corasick
Manacher's algorithm
Letter position lists
Combinatorial search
Meet in the middle
Brute-force with pruning
Best-first (A*)
Bidirectional search
Iterative deepening DFS / A*
Data structures
LCA (2^k-jumps in trees in general)
Pull/push-technique on trees
Heavy-light decomposition
Centroid decomposition
Lazy propagation
Self-balancing trees
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
Monotone queues / monotone stacks / sliding queues
Sliding queue using 2 stacks
Persistent segment tree